



Algoritmi e strutture dati

ASD2023_II_Programmazione [AULA]



YIMING HOU

297226

Iniziato martedì, 21 febbraio 2023, 10:00

Terminato martedì, 21 febbraio 2023, 11:40

Tempo impiegato 1 ora 40 min.

Informazione

PER ENTRAMBE LE PROVE DI PROGRAMMAZIONE (18 o 12 punti)

- se non indicato diversamente, è consentito utilizzare chiamate a funzioni standard, quali ordinamento per vettori, funzioni su FIFO, LIFO, liste, BST, tabelle di hash, grafi e altre strutture dati, considerate come librerie esterne
- gli header file riferiti dal codice dovranno essere inclusi nella versione del programma allegata alla relazione
- i modelli delle funzioni ricorsive **non** sono considerati funzioni standard
- consegna delle relazioni, per entrambe le tipologie di prova di programmazione: entro VENERDI' 24/02/2023, alle ore 14:00, mediante caricamento su Portale
- **SOLO LAUREANDI che avessero bisogno di una correzione anticipata** consegna delle relazioni, per entrambe le tipologie di prova di programmazione: entro MERCOLEDI' 22/02/2023, alle ore 14:00, mediante caricamento su Portale
- le istruzioni per il caricamento sono pubblicate sul Portale nella sezione Materiale
- **QUALORA IL CODICE CARICATO CON LA RELAZIONE NON COMPILI CORRETTAMENTE, VERRÀ APPLICATA UNA PENALIZZAZIONE.** Si ricorda che la valutazione del compito viene fatta, senza la presenza del candidato, sulla base dell'elaborato svolto durante l'appello. Non verranno corretti i compiti di cui non sarà stata inviata la relazione nei tempi stabiliti.

Informazione

Attenzione!

Indicare quale tipologia di esame si intenda svolgere rispondendo alla domanda a scelta multipla seguente (domanda 1).

Si noti che è possibile leggere tutte o parte delle domande prima di effettuare la scelta e rispondere alla domanda 1.

Le domande dalla 2 alla 6 sono dedicate all'esame semplificato (traccia da 12pt).

In particolare:

- la domanda 2 fa parte dell'esercizio da 2 punti.
- la domanda 3 fa parte dell'esercizio da 4 punti.
- le domande 4,5,6 fanno parte dell'esercizio da 6 punti.

Le domande dalla 7 in poi sono dedicate all'esame completo (traccia da 18pt).

Un apposito separatore fa da intervallo tra le domande del compito da 12pt e le domande del compito da 18pt.

Domanda 1

Risposta non data

Non valutata

Indicare quale tipologia di esame si intende svolgere.

- ☐ (a) 12 Punti - Semplificato
- ☐ (b) 18 Punti - Completo

Domanda 2

Risposta non data

Punteggio max.: 2,00

Sia data una matrice M di dimensione r x c contenente singoli caratteri minuscoli.

Scrivere una funzione che generi una matrice M' di dimensioni opportune derivata da M mantenendo solo le righe/colonne dove non sia presente alcuna vocale {a, e, i, o, u}.

La matrice M' sia allocata dentro alla funzione.

Completare opportunamente il prototipo in modo che la nuova matrice e le rispettive dimensioni siano disponibili al chiamante.

```
void f(char **M, int r, int c, ...);
```

Esempio

$$M = \begin{pmatrix} a & c & f & e & g \\ z & y & t & t & p \\ q & w & j & e & t \\ p & l & l & n & m \end{pmatrix} \rightarrow M' = \begin{pmatrix} y & t & p \\ l & l & m \end{pmatrix}$$

Nota

Potenzialmente la funzione potrebbe dover ritornare una matrice nulla, di dimensione 0 x 0.

Domanda 3

Risposta non data

Punteggio max.: 4,00

Definire una struttura dati adeguata a rappresentare una lista doppio linkata di interi come ADT I classe, e il relativo nodo.

Il tipo lista si chiami LIST.

La lista definita non deve fare uso di sentinelle.

Indicare esplicitamente in quale modulo/file appare la definizione dei tipi proposti.

Usando i tipi precedentemente definiti si scriva una funzione void f(LIST l, int a, int b) che elimini dalla lista tutti i nodi il cui valore sia compreso tra a e b, estremi inclusi.

Non è ammesso l'uso di funzioni di libreria.

Domanda 4

Risposta non data

Punteggio max.: 6,00

Una macchina industriale può produrre pezzi di diversi tipi, ognuno caratterizzato da un valore e da un tempo di produzione, valori interi e positivi.

Si assuma per comodità che i vari tipi di pezzi siano univocamente identificati da un singolo intero nell'intervallo $0 \dots P-1$.

Ricevuto in input un tempo T e un valore obiettivo V , entrambi interi e positivi, identificare quanti pezzi di ogni tipo sia possibile produrre entro il tempo T per minimizzare la differenza, in valore assoluto, tra V e la somma dei valori dei pezzi prodotti, ossia per avvicinarsi il più possibile all'obiettivo.

Domanda 5

Risposta non data

Non valutata

Si giustifichi la scelta del modello combinatorio adottato.

Domanda 6

Risposta non data

Non valutata

Si descrivano i criteri di pruning adottati o il motivo della loro assenza.

PAGINA DI INTERMEZZO

La prova da 12 punti termina prima di questa pagina di intermezzo.

La prossima domanda rappresenta l'inizio della prova da 18 punti.

Descrizione del problema

Un'azienda deve pianificare l'assegnazione del proprio personale a una serie di incarichi.

L'azienda ha a disposizione P persone da suddividere tra T incarichi. Ogni persona deve essere assegnata a un singolo incarico. Per comodità si assuma che le persone siano identificate da un indice intero nel range $0 \dots P-1$ e gli incarichi da un indice nel range $0 \dots T-1$.

L'azienda valuta la difficoltà e il valore di ogni incarico con un singolo valore intero d_i . Tali valori sono memorizzati in un vettore D . Ogni dipendente dell'azienda ha un proprio livello di esperienza personale e_i . Tali valori sono memorizzati in un vettore E .

Tra il personale dell'azienda si instaurano anche delle sinergie, mappate in una matrice quadrata simmetrica S di dimensioni $P \times P$. Ogni cella $S[i][j]$ della matrice rappresenta il contributo dato da avere il dipendente i e il dipendente j assegnati al medesimo incarico.

Perché un incarico possa essere svolto con successo occorre assegnare personale tale per cui la somma dell'esperienza individuale e i contributi dovuti alle sinergie raggiungano almeno il 75% del rispettivo valore d_i . Se non è possibile raggiungere tale soglia per un certo incarico, il contributo di tutta la forza lavoro assegnata è sostanzialmente perduto. È possibile assegnare più personale del dovuto a un certo incarico, ma tutto il contributo oltre il valore d_i dell'incarico stesso è irrilevante.

La resa di ogni incarico svolto con successo è quindi data dal minimo tra il valore complessivo della forza lavoro assegnata (individuale e sinergica) e il valore d_i dell'incarico stesso. Data una assegnazione di persone ai vari incarichi, il valore complessivo dell'assegnazione corrisponde alla somma delle rese dei singoli incarichi svolti con successo.

Esempio

$D = \{10, 6, 8\}$ // Vettore degli incarichi

$E = \{3, 2, 5, 3\}$ // Vettore dell'esperienza delle persone

$S =$ // Matrice di sinergia

```
{  
  { 0, 2, 1, 4 },  
  { 2, 0, 4, 1 },  
  { 1, 4, 0, 3 },  
  { 4, 1, 3, 0 }  
}
```

Assegnando le persone di indice 0 e 3 all'incarico di indice 0 si riuscirebbe a soddisfare completamente il valore d_0 con resa pari a 10 (3+3 individuali e +4 di sinergia).

Assegnando le persone di indice 0 e 1 all'incarico di indice 0 si arriverebbe a un contributo lavorativo pari a 7 (3+2 individuali e +2 di sinergia) che non permette di raggiungere il 75% richiesto, per cui la resa è completamente nulla.

Assegnando le persone di indice 0 e 2 all'incarico di indice 0 si arriverebbe a un contributo lavorativo pari a 9 (3+5 individuali e +1 di sinergia) che permette di svolgere l'incarico con resa pari a 9.

Assegnando le persone di indice 0 e 3 all'incarico di indice 1 si riuscirebbe a soddisfare (in esubero) completamente il valore d_1 , con resa pari a 6.

Richieste del problema

A seguire una sintesi delle richieste del problema. Per ogni richiesta si troverà una domanda dedicata nelle sezioni a seguire con una descrizione più dettagliata per le richieste.

Strutture dati

Definire opportune strutture dati per rappresentare le informazioni presentate in precedenza e quanto sia ritenuto necessario alla soluzione dei problemi di verifica e di ricerca. Acquisire i dati nelle strutture definite.

Problema di verifica

Data una proposta di assegnazione completa persone-incarichi, valutare che questa rispetti i dati del problema e in caso affermativo valuti la resa complessiva dell'assegnazione proposta, secondo le regole illustrate in precedenza.

Problema di ottimizzazione

Identificare una assegnazione tale da massimizzare la resa complessiva.

Domanda 7

Completo

Non valutata

Strutture dati e acquisizione

Definire opportune strutture dati per rappresentare le informazioni che caratterizzano il contesto e quanto sia ritenuto necessario alla soluzione dei problemi di verifica e di ricerca.

Scrivere qui la definizione e implementazione delle strutture dati repute necessarie a modellare le informazioni del problema secondo quanto richiesto e le funzioni di acquisizione dei dati stessi. In caso di organizzazione delle strutture dati su più file, indicare esplicitamente il modulo di riferimento.

Si assuma che le informazioni relative al problema siano riportate in un unico file input.txt con il seguente formato:

- Sulla prima riga è riportata il numero T di incarichi.
- La seconda riga contiene T valori interi separati da un singolo spazio a rappresentare il valore d di ogni incarico
- Sulla terza riga è riportata il numero P di persone.
- La quarta riga contiene P valori interi separati da un singolo spazio a rappresentare il valore e di ogni persona
- Seguono P righe composte da P interi separati da un singolo spazio, ciascuna, a rappresentare i valori della matrice S

Rispetto all'esempio presentato in precedenza, il contenuto del file corrispondente sarebbe il seguente (*i commenti appaiono solo come ulteriore chiarimento. non fanno parte del file!*):

```
3 // T
10 6 8 // Valori associati ai T = 3 incarichi
4 // P
3 2 5 3 // Esperienza delle P = 4 persone
0 2 1 4 // Prima riga della matrice S (4x4)
2 0 4 1
1 4 0 3
4 1 3 0 // Ultima riga della matrice S
```

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

```
...
a.h
typedef struct inc_s{int T, int *tv, int P; int *pv; int **mad}} *inc;
typedef struct assign_s{int n, int *pvs, int resa} assigns;
typedef struct sol_s{int T, assigns *assign, int N} *sol;

a.c

inc leggifile(FILE *in){
inc G;
int i, j;
fscanf(in, "%d", &(G->T));
G->tv=malloc(G->T*sizeof(int));
for(i=0; i<G->T; i++){fscanf(in, "%d", &(G->tv[i]));}
fscanf(in, "%d", &(G->P));
G->pv=malloc(in, "%d", &(G->pv[i]) );
```

```

G->adj=malloc2d( G->P, G->P, -1);
for(i=0; i<G->P; i++){fscanf(in, " %d", &(G->pv[i]) );}
for(i=0; i<G->P; i++){
    for(j=0; j<G->P; j++;
        fscanf(in, " %d" , &(G->adj[i][j]));
    }
return G;
}

static int malloc2d(int R, int C, int val){
int **mat;
int i,j;
mat=malloc(R*sizeof(int*));
for(i=0; i<R; i++){
    *mat=malloc(C*sizeof(int));
}
for(i=0; i<R; i++)
    for(j=0; j<C; j++)
        mat[i][j]=val;
return mat;
}

sol getproposta(FILE *in ){
sol ris; int i, j;
ris=malloc(sizeof(struct sol_s));
fscanf(in, " %d %d ", &(ris->T), &(ris->N));
sol->assign=malloc(ris->T*sizeof(assigns));
for(i=0; i<ris->T; i++){
    fscanf(in , "%d", &(ris->assign[i].n));
    ris->assign[i]->pvs=malloc(ris->assign[i].n*sizeof(int));
    ris->assign[i].resa=0;
    for(j=0; j<ris->assign[i].h; j++){
        fscanf(in, " %d" , &(ris->assign[i].pvs[j]));}
}
return ris;
}

```

Domanda 8

Completo

Non valutata

Problema di verifica

Data una proposta di assegnazione, valutare che questa rispetti i dati del problema e in tal caso determinare il valore della resa complessiva corrispondente.

Il formato dei dati in input per questa richiesta è a discrezione del candidato, **che è tenuto a fornirne anche una breve descrizione.**

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

...input: numero di incarichi T numero di persone N occupate

T righe dove in ogni riga è presente:

numero di persone assegnato a quel incarico

#una lista di indici che indicano quelle persone

si parte dal presupposto che la proposta sia relativo al file input.txt e che la matrice delle sinergie sia salvata, anche ris e G siano entrambi salvati già nelle strutture dati dopo aver usato le funzioni di inserimento (sin è la matrice per comodità in scrittura)

```
int check(int **sin, int G, sol ris){ //ritorna 0 se non è accettabile anche se vengono stampata man ma no le soluzioni parziali
```

```
int *flag=calloc(ris->N, sizeof(int));
```

```
int sum=0;
```

```
int i,j, k;
```

```
if(ris->T != G->T) return 0; //numero incarichi non conforme all'input
```

```
if(ris->N != G->P) return 0; //numero di persone assegnate non conforme all'input (ci sono persone che non fanno niente o ci si stanno usando più persone di quante ce ne sono realmente disponibili
```

```
for(i=0; i< ris->T; i++){
```

```
    for(j=0; j<ris->assign[i].n; j++){
```

```
        if(flag[ris->assign[i].pvs[j]]==1){ return 0; } //sto tentando di assegnare una persona a più incarichi
```

```
        ris->assign[i].resa +=G->pv[ris->assign[i].pvs[j]];
```

```
        flag[ris->assign[i].pvs[j]]=1;
```

```
        if(j>0) //aggiunta dei valori nella matrice delle sinergie
```

```
            ris->assign[i].resa+=sin[ris->assign[i].pvs[j-1]][ris->assign[i].pvs[j]];
```

```
        //se il valore supera il 75% del valore dell'incarico
```

```
        if((float)ris->assign[i].resa>= (float)75* (float)(G->tv[i])/(float)100) //confronto tra float ma valori salvati e mostrati rimangono int
```

```
            printf("resa assignment %d è di %d, i, ris->assign[i].resa);
```

```
        else { ris->assign[i].resa=0; printf("resa vale zero ")}
```

```
        sum+=ris->assign[i].resa;
```

```
    }
```

```
return sum;
```

```
}
```

Domanda 9

Completo

Non valutata

Problema di ricerca e ottimizzazione

Identificare una assegnazione tale da massimizzare la resa complessiva secondo le regole illustrate in precedenza, ovvero:

Perché un incarico possa essere svolto con successo occorre assegnare personale tale per cui la somma dell'esperienza individuale e i contributi dovuti alle sinergie raggiungano almeno il 75% del valore rispettivo valore d_i . Se non è possibile raggiungere tale soglia per un certo incarico, il contributo di tutta la forza lavoro assegnata è sostanzialmente perduto. È possibile assegnare più personale del dovuto a un certo incarico, ma tutto il contributo oltre il valore d_i dell'incarico stesso è irrilevante.

La resa di ogni incarico svolto con successo è quindi data dal minimo tra il valore complessivo della forza lavoro assegnata (individuale e sinergica) e il valore d_i dell'incarico stesso. Data una assegnazione di persone ai vari incarichi, il valore complessivo dell'assegnazione corrisponde alla somma delle rese dei singoli incarichi svolti con successo.

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

```
...
void solve(inc G){
    int i, best=0;
    int *rr, *tmpr, *flag;
    rr=malloc(G->P*sizeof(int));
    tmpr=malloc(G->P*sizeof(int));
    flag=calloc(G->P, sizeof(int));
    //allocazione per ris
    sol ris=malloc(sizeof(struct sol_s));
    ris->assign=malloc(G->T*sizeof(assigns));
    //sovralloco e rialloco alla fine
    for(i=0; i<G->T; i++){
        ris->assign[i]=malloc(G->P*sizeof(int));
    }

    solve_r(0,G->T, rr, tmpr, &best, 0, G->P, G, flag );
    ..salvataggio in ris...
    ..riallocazione di ris..
}

int calcolareas(int **sin, inc G, int *sol){
    int *resa, i,j=0, k=0, s, l, totresa=0 ;
```

```

int *vectsin; //
vectsin=calloc(G->P, sizeof(int));
resa=calloc(G->T, sizeof(int));
for(i=0; i<G->P, i++){
    resa[sol[i]]+=G->pv[i];
    if(sol[i]==0){
        vectsin[k]=i; k++; //salvo gli indici di elementi che lavorano in stesso progetto
    }
}
//aggiungo il valore dato dalle sinergie
for(s=1; s<k; s++) //n-1 volte
    resa[0]+=sin[vectsin[k-1]][vectsin[k]]; //sinergia precedente con quello corrente
for(i=1; i<G->T; i++){
    //per ogni progetto salvo in un vettore gli indici delle persone che ci lavorano (da 1 dato che ho già la r
    esa totale del pprogetto 0)
    //per n-1 con n numero di persone che lavorano sullo stesso progetto
    //sommo la sinergia delle due persone successive uno all'altro
    k=0;
    for(j=0; j<G->P; j++){
        vectsin[i]=j; k++;
    }

    for(s
}

for(i=0; i<G->T; i++){if((float)resa[i]>=(float) 75*G->pv[i]/(float)100) totresa+=resa[i];}
return totresa;
}

void copyvect(int *dest, int *source, int n){
int i=0;
for(i=0; i<n; i++){
    dest[i]=source[i];
}
}

//disposizioni ripetute usando i task
void solve_(int pos, int n, int* rr, int *tmpr, int *best, int curmax, int k, inc G , int *flag){
int i;
if(pos>=k){ //è stato usato un vettore di interi rendendo la funzione check del punto precedente inadatt
o a controllare
//se la soluzione è accettabile o meno, si usa calcolaresa, ma l'idea alla base, anche con strutture dati
differenti, è la stessa
    curmax=calcolaresa(G->madj, G, tmpr);
    if(curmax> *best){

```

```
*best=curmax;
copyvect(rr, tmpr, k);
}
return;
}
for(i=0; i<n; i++){
    tmpr[pos]=i;
    solve_r(pos+1, n, rr, tmpr, best, curmax, k ,G, flag);
}
}
```